



Context, Activity e Intents

El contexto de Android

El contexto de la aplicación es un objeto que permite disponer de una interfaz a la información global de un entorno de ejecución.

Se trata de una clase abstracta cuya implementación es proporcionada por el sistema Android.

Permite a la aplicación acceder a recursos y clases específicas de la aplicación, así como realizar llamadas al sistema para operaciones a nivel de aplicación, como lanzar actividades, generación de intents, etc...

En otras palabras, Context representa el estado actual de la aplicación y permite obtener información de su entorno de ejecución, acceder a recursos de la aplicación y efectuar operaciones, como lanzar actividades, entre otras.

Cada vez que se inicie de la aplicación se creará un contexto nuevo. En caso de que la aplicación pierda el foco, pero lo vuelva a recuperar sin ser destruida la aplicación, el contexto será el mismo.

¿Qué ocurre si almacenaremos el contexto en una variable local e intentaremos utilizarla más tarde?

Activities

Para navegar de una a otra, podemos utilizar un objeto intent. Veámoslo con un ejemplo: vamos a crear una nueva actividad en nuestra app. En el menú de Android Studio, seleccionamos File > New > Activity > Empty Activity. El wizard añadirá no solo una clase MainActivity2, sino también su layout y la línea en el manifiesto. Para navegar de nuestra actividad a la nueva, modificaremos el código de este modo:

*crea una segunda actividad

```
class MainActivity : AppCompatActivity() {
    private lateinit var boton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        boton = findViewById(R.id.boton)
        boton.setOnClickListener {
            val intent = Intent(this, SegundaActividad::class.java)
            startActivity(intent)
        }
    }
}
```



```
}  
}  
}
```

Creamos un objeto Intent: el primer parámetro es el contexto de nuestra actividad. El segundo parámetro es la clase de la activity a la que deseamos navegar.

Una vez creado el Intent, llamaremos a la función de la startActivity con él y la nueva actividad se mostrará sobre la actual. Decimos "sobre" porque el sistema Android guardará una pila de vistas una sobre otra según vayamos abriéndose, lo que se conoce como backstack. Cuando cerremos una ventana, presionando Atrás, por ejemplo, se mostrará la anterior, la que está debajo en la pila.

Cada actividad en Android tiene su propio ciclo de vida y gestiona sus propios recursos. El Context actúa como un enlace entre la actividad y el sistema operativo Android para acceder a los recursos, inflar vistas y realizar otras operaciones que dependen del entorno de la aplicación.

Pasar datos a actividades iniciadas

Además de especificar la actividad que queremos mostrar, una intención también puede pasar datos clave-valor entre actividades. Similar a los métodos GET y POST para una solicitud HTTP. Puede especificar los parámetros colocando pares clave-valor en el Intent con la método putExtra()

Devolver el resultado de los datos a la actividad principal

En el caso típico de usar startActivity, la actividad se inicia y se agrega a la pila de navegación y no se espera ningún resultado. Si el usuario desea cerrar la actividad, simplemente puede presionar "atrás" y se mostrará la actividad principal.

En ocasiones se desea que la actividad iniciada devuelva un resultado cuando finalice. En este caso, usamos un *ActivityResultLauncher*, que inicia una actividad y espera que *Result* regrese.



```
class MainActivity : AppCompatActivity() {

    private lateinit var boton: Button
    private lateinit var startForResult:
ActivityResultLauncher<Intent> // Declarar fuera de onCreate

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        boton = findViewById(R.id.boton)

        // Registrar el ActivityResultLauncher fuera del
onClickListener
        startForResult = registerForActivityResult(
            ActivityResultContracts.StartActivityForResult()
        ) { result ->
            if (result.resultCode == Activity.RESULT_OK) {
                val data = result.data
                if (data != null) {
                    val returnString =
data.extras?.getString("stringdevuelta")
                    Log.d("intents", "Resultado devuelto:
$returnString")
                }
            }
        }
        // Listener del botón
        boton.setOnClickListener {
            val intent = Intent(this, SegundaActividad::class.java)
            intent.putExtra("usuario", "yo")
            startForResult.launch(intent) // Lanzar la actividad
        }
    }
}
```

Esto iniciará la subactividad y, cuando la subactividad esté completa, podrá devolver el resultado a la actividad padre.

En este caso, en nuestra segunda actividad debemos de añadir los clave-valor deseados.



```
var dato=intent.getStringExtra("usuario")
Log.i("intents", "DATos: $dato")

val data = Intent()
data.putExtra("stringdevuelta", "he devuelto un valor")
setResult(RESULT_OK, data)
finish()
```

El parámetro resultCode: normalmente será una de los valores RESULT_OK (para indicar que la actividad terminó adecuadamente y pueden utilizarse los valores devueltos) y RESULT_CANCELED (si el usuario canceló el proceso y debemos tomar las medidas adecuadas ante el rechazo del proceso).

Implicit

Intents

(<https://developer.android.com/guide/components/intents-common?hl=es-419>)

Las intenciones implícitas son solicitudes para realizar una acción basada en una acción deseada y datos objetivo. Esto contrasta con una intención explícita que apunta a una actividad específica. Por ejemplo, si quiero hacer una llamada telefónica para el usuario, puedo hacerlo con esta intención:

Si quiero iniciar un sitio web en el navegador del teléfono, puedo hacer esto:

```
boton.setOnClickListener {
    val browserIntent = Intent(Intent.ACTION_VIEW,
"https://www.xataka.com".toUri())
    if (browserIntent.resolveActivity(packageManager) != null) {
        startActivity(browserIntent)
    }
}
```

Con el nivel de API 30, si apunta a esa versión o una superior, su aplicación no puede ver ni interactuar directamente con la mayoría de los paquetes externos sin solicitar explícitamente



que se le permita. Necesitamos ese permiso aquí porque queremos encontrar una aplicación instalada que pueda ver las URL, normalmente un navegador web.

```
<queries>
  <intent>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.BROWSABLE"
  />

    <data android:scheme="http" />
  </intent>
</queries>
```

Otro ejemplo sería abrir el marcador del teléfono con un número premarcado:

```
val callIntent = Intent(Intent.ACTION_DIAL)
callIntent.data = "tel:999555222".toUri()
if(callIntent.resolveActivity(packageManager) != null) {
    startActivity(callIntent)
} else {
    Log.e("MainActivity", "No hay aplicación instalada para manejar
la acción.")
}
```

Y su query en el manifest:

```
<queries>
  <intent>
    <action android:name="android.intent.action.DIAL" />
  </intent>
</queries>
```

Lo realmente importante aquí es que creamos un intent de forma diferente a como hemos visto hasta ahora, y es porque estamos accediendo a una activity que está fuera de nuestra app.

En ocasiones, para realizar ciertas tareas se requiere pedir permiso al usuario, ya que de no ser así, podríamos violar la privacidad del usuario o realizar acciones potencialmente peligrosas (acceder a los contactos, tomar el control de la cámara, realizar llamadas, etc). Para pasar los controles de seguridad de Android el sistema se asegurará primero de que el usuario nos dé su consentimiento.

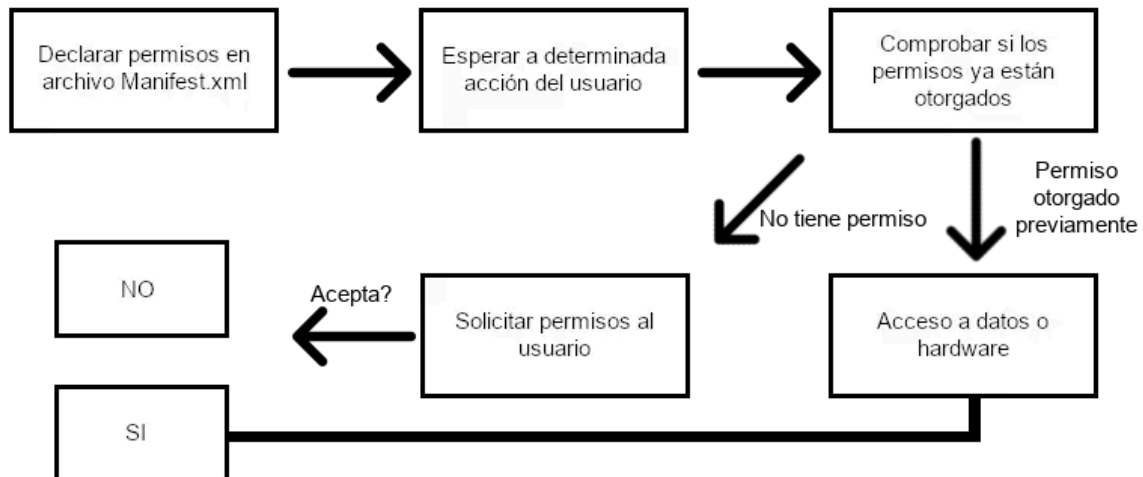


Algunos permisos son menos importantes que otros. Los permisos normales basta con pedirlos en el manifest para que, al instalar, el usuario sepa que la app accederá a esos datos.

Los permisos más peligrosos deberán especificarse tanto en el manifest como a la hora de ejecutar la app.

Categoría	Permiso	Descripción
Permisos Normales	INTERNET	Permite el acceso a Internet.
	ACCESS_NETWORK_STATE	Permite verificar el estado de las conexiones de red.
	FLASHLIGHT	Permite el uso de la linterna del dispositivo.
	VIBRATE	Permite el uso del vibrador del dispositivo.
Permisos Peligrosos	ACCESS_FINE_LOCATION	Permite acceder a la ubicación precisa del usuario.
	READ_CONTACTS	Permite leer los contactos del usuario.
	CALL_PHONE	Permite realizar llamadas telefónicas sin intervención del usuario.
	CAMERA	Permite acceder a la cámara del dispositivo.

Las declaraciones de permisos en el archivo Manifest.xml son de lo más sencillas. Basta con incluir una nueva línea dentro del primer nivel de las etiquetas <manifest>. En esta nueva línea hay que incluir la etiqueta <uses-permission /> en la que añadiremos el valor «android:name» seguido del permiso a declarar. Os dejo por aquí el enlace a [todos los permisos de Android](#). Como podréis ver, hay muchísimos



Este sistema de permisos en tiempo de ejecución es una mejora de seguridad crítica introducida en **Android 6.0 (API 23, Marshmallow)** para dar a los usuarios un mayor control sobre su privacidad.

```

class MainActivity : AppCompatActivity() {
    private lateinit var boton: Button
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        //bind
        boton = findViewById(R.id.boton)

        // Listener del botón
        boton.setOnClickListener {
            if (tenemosPermiso(Manifest.permission.CALL_PHONE)) {
                realizarLlamada()
            } else {
                solicitarPermiso(Manifest.permission.CALL_PHONE)
            }
        }
    }
}

private fun tenemosPermiso(permiso: String): Boolean {

```



```
        return ContextCompat.checkSelfPermission(this, permiso) ==
PackageManager.PERMISSION_GRANTED
    }

    /**
     * Indica el permiso que queremos solicitar y quién es el
responsable de gestionar la solicitud
     * de permisos en tiempo de ejecución.
     *
     * @param permiso El permiso a solicitar (ej:
Manifest.permission.CAMERA).
     */
    private fun solicitarPermiso(permiso: String) {
        //Aquí, this indica quien es el responsable de gestionar la
solicitud de permisos y requestCode es un identifficador para
nosotros
        ActivityCompat.requestPermissions(this, arrayOf(permiso), 1)
    }

    private fun realizarLlamada() {
        val numeroTelefono = "tel:123456789" // Número de teléfono
a llamar
        val intent = Intent(Intent.ACTION_CALL)
        intent.data = numeroTelefono.toUri()
        startActivity(intent)
    }

    /**
     * Metodo de callback invocado cuando el usuario responde a un
diálogo de solicitud de permisos
     * iniciado por ActivityCompat.requestPermissions.
     *
     * Se utiliza para determinar si el permiso fue concedido o
denegado,
     * basándose en el código de solicitud único (requestCode).
     *
     * @param requestCode El código de solicitud (entero) pasado
originalmente a
     * requestPermissions(). Usado aquí para identificar la solicitud.
     * @param permissions La lista de permisos solicitados. Nunca es
nula.
    */
}
```




```
* @param grantResults Los resultados de la concesión (GRANTED o
DENIED)
* correspondientes a los permisos. Nunca es nula.
* PERMISSION_GRANTED = 0, PERMISSION_DENIED = -1.
*/
override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<out String>,
    grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions,
grantResults)
    if (requestCode == 1) {
        if (grantResults.isNotEmpty() && grantResults[0] ==
PackageManager.PERMISSION_GRANTED) {
            realizarLlamada()
        } else {
            Toast.makeText(this, "Permiso de llamada denegado",
Toast.LENGTH_SHORT).show()
        }
    }
}
}
```

En nuestro manifest:

```
<uses-feature
    android:name="android.hardware.telephony"
    android:required="false" />
<uses-permission android:name="android.permission.CALL_PHONE"/>
```